	UNIVERSIDADE DE FORTALEZA – UNIFOR – FORTALEZA/CE	
Responsável: Mário Eduardo - 2316157 Luiz Eduardo - 2310314 Gregory Jereissati - 2320425	Projeto: **HealthSys Distribuído**	Código: T335-16
Formulário: **Especificação de Requisitos de Software**		Form. Versão: **1.0**

Especificação de Requisitos de Software

HealthSys Distribuído

Histórico de versões

Data	Versão	Descrição	Autor
01/04/2026	1.0	Elaboração inicial do SRS do sistema HealthSys Distribuído.	Equipe do projeto

Sumário

1. Introdução 4
   - 1.1 Finalidade 4
   - 1.2 Escopo do Produto 4
   - 1.3 Referências 4

2. Descrição geral 4
   - 2.1 Perspectiva do produto 4
   - 2.2 Funções do produto 4
   - 2.3 Classes de usuários e Características 4
   - 2.4 Ambiente de trabalho 5
   - 2.5 Design e Implementação Restrições 5
   - 2.6 Documentação do usuário 5
   - 2.7 Suposições e Dependências 5

3. Requisitos do Sistema 5
   - 3.1 Requisitos Funcionais 5
   - 3.2 Requisitos Não Funcionais 5
   - 3.3 Regras de Negócio 5

4. Casos de Uso 6
   - 4.1 Atores 6
   - 4.2 Casos de Uso 6

5. Apêndice A: Glossário 6

6. Apêndice B: Modelos de Análise 6

7. Aprovação 7

1. Introdução

Este documento contempla os requisitos necessários para o desenvolvimento do HealthSys Distribuído, sistema proposto no contexto da disciplina de Computação Distribuída, cursada no 7º período, que visa apoiar hospitais e unidades de saúde no gerenciamento de pacientes, prontuários eletrônicos e triagens médicas. A proposta do sistema é oferecer uma plataforma distribuída capaz de integrar diferentes funcionalidades hospitalares de forma escalável, segura e tolerante a falhas, utilizando uma arquitetura baseada em microservices.

Por meio dessa solução, busca-se melhorar a organização dos processos hospitalares, facilitar o acesso às informações clínicas, otimizar a comunicação entre profissionais de saúde e dar suporte à realização de teletriagens e acompanhamentos médicos. Além disso, o projeto permite aplicar, na prática, conceitos fundamentais da computação distribuída, como comunicação entre serviços, alta disponibilidade, escalabilidade horizontal, mensageria e monitoramento de sistemas distribuídos.

1.1 Finalidade

A finalidade deste documento é identificar, descrever e organizar os requisitos funcionais e não funcionais necessários ao planejamento e ao desenvolvimento do sistema HealthSys Distribuído. Os requisitos funcionais correspondem às funcionalidades e aos serviços que o sistema deverá oferecer aos seus usuários, descrevendo as ações e operações esperadas da plataforma. Já os requisitos não funcionais dizem respeito às características que definem a qualidade e o comportamento do sistema, como desempenho, segurança, escalabilidade, disponibilidade e confiabilidade, não estando necessariamente detalhados diretamente nos casos de uso.

1.2 Escopo do Produto

O HealthSys Distribuído é uma plataforma de gestão hospitalar distribuída desenvolvida com a finalidade de apoiar hospitais e unidades de saúde no gerenciamento de pacientes, prontuários eletrônicos e triagens médicas. O sistema foi concebido para centralizar e organizar informações clínicas relevantes, permitindo maior eficiência no atendimento, melhor acompanhamento dos pacientes e apoio à comunicação entre os profissionais de saúde.

A solução proposta contempla funcionalidades como cadastro e gestão de pacientes, consulta de histórico, registro de prontuário eletrônico, avaliação de sintomas, classificação de risco e teletriagem médica, além de relatórios e notificações voltadas ao ambiente hospitalar. Sua arquitetura é baseada em microservices distribuídos, com integração entre serviços e uso de tecnologias que favorecem escalabilidade, tolerância a falhas e comunicação assíncrona.

Entre os principais benefícios esperados com o desenvolvimento do sistema estão a melhoria da organização dos processos hospitalares, a otimização do fluxo de informações, o suporte à tomada de decisão clínica e o aumento da eficiência operacional no atendimento à saúde. Além disso, o projeto tem como objetivo aplicar, de forma prática, os conceitos estudados na disciplina de Computação Distribuída, tais como arquitetura distribuída, comunicação entre serviços, alta disponibilidade, escalabilidade horizontal, mensageria e monitoramento de sistemas.

Dessa forma, o HealthSys Distribuído busca aliar os objetivos acadêmicos da disciplina ao desenvolvimento de uma solução tecnológica capaz de contribuir para a modernização da gestão hospitalar, demonstrando como os fundamentos da computação distribuída podem ser utilizados em aplicações reais na área da saúde.

1.3 Referências

HEALTHSYS DISTRIBUÍDO. Projeto Final – Gestão Hospitalar. Documento-base do projeto acadêmico, contendo visão geral, arquitetura, funcionalidades, infraestrutura, cronograma e tecnologias previstas para o desenvolvimento do sistema. 2026. Arquivo em PDF disponibilizado para a disciplina.

IEEE. IEEE/ISO/IEC 29148-2018 – Systems and software engineering — Life cycle processes — Requirements engineering. New York: IEEE Standards Association, 2018. Norma de referência para engenharia de requisitos e especificação de requisitos de software.

SPRING. Spring Boot Reference Documentation. Documentação oficial do Spring Boot, utilizada como referência para a implementação dos serviços de backend e da arquitetura baseada em microservices. Spring, 2026.

DOCKER. Docker Documentation. Documentação oficial utilizada como referência para containerização dos serviços e execução do ambiente distribuído. Docker Inc., 2026.

KUBERNETES. Kubernetes Documentation. Documentação oficial utilizada como referência para orquestração, gerenciamento e escalabilidade dos containers da aplicação. Kubernetes Project, 2026.

2. Descrição Geral

2.1 Perspectiva do Produto

O **HealthSys Distribuído** é um sistema novo, proposto no contexto da disciplina de **Computação Distribuída**, com a finalidade de apoiar hospitais e unidades de saúde no gerenciamento de pacientes, prontuários eletrônicos e triagens médicas. Trata-se de uma plataforma autossuficiente, concebida desde sua origem para operar com uma arquitetura distribuída baseada em microservices, buscando oferecer escalabilidade, tolerância a falhas e melhor organização dos processos hospitalares.

A solução é composta por um **frontend cliente**, responsável pela interação com os usuários do sistema, e por um conjunto de **serviços distribuídos no backend**, cada um com responsabilidades específicas. Entre esses componentes estão o **API Gateway**, o **Serviço de Pacientes**, o **Serviço de Prontuário**, o **Serviço de Triagem**, o **Serviço de Usuários** e o **Serviço de Notificações**, que se comunicam por meio de mecanismos síncronos e assíncronos. Essa organização permite a separação de responsabilidades e a integração entre os diferentes módulos que compõem o sistema.

No contexto geral, o produto não é apresentado como substituição direta de um sistema legado específico, mas sim como uma proposta de plataforma moderna para gestão hospitalar distribuída. Seu desenvolvimento também está alinhado aos objetivos acadêmicos da disciplina, uma vez que aplica conceitos como arquitetura cliente-servidor, comunicação entre serviços, uso de mensageria, alta disponibilidade, escalabilidade horizontal, virtualização e observabilidade.

De forma geral, o sistema pode ser entendido como um produto maior formado por subsistemas interligados. O frontend se comunica com o **API Gateway**, que centraliza o roteamento das requisições para os serviços internos. Esses serviços, por sua vez, interagem com bancos de dados específicos e mecanismos de comunicação assíncrona, como **Kafka** e **RabbitMQ**, além de recursos de infraestrutura como **Redis**, **Docker** e **Kubernetes**. Assim, o HealthSys Distribuído se insere como uma solução distribuída completa, na qual diferentes componentes cooperam para fornecer as funcionalidades esperadas no ambiente hospitalar.

2.2 Funções do Produto

O sistema **HealthSys Distribuído** terá como principais funções:

- realizar o **cadastro e a gestão de pacientes**;
- permitir a **consulta de histórico do paciente**;
- registrar informações clínicas no **prontuário eletrônico distribuído**;
- armazenar dados de **consultas, exames e medicamentos**;
- registrar informações complementares, como **alergias e vacinas**;
- permitir a realização de **teletriagem médica**;
- apoiar a **avaliação de sintomas** e a **classificação de risco** dos pacientes;
- possibilitar a **visualização de prontuários** e relatórios médicos;
- oferecer **comunicação entre profissionais de saúde**;
- disponibilizar um **dashboard hospitalar** para acompanhamento das informações do sistema;
- realizar o **controle de acesso e autenticação de usuários**;
- enviar **notificações e alertas** relacionados às atividades da plataforma.

2.3 Classes de Usuários e Características

Considerando as funcionalidades previstas para o **HealthSys Distribuído**, as classes de usuários do sistema podem ser identificadas de acordo com o tipo de interação com a plataforma, o nível de acesso às informações e as responsabilidades desempenhadas no contexto hospitalar. As classes de usuários mais relevantes para o sistema são descritas a seguir.

a) Administradores do sistema

Correspondem aos usuários responsáveis pelo gerenciamento técnico e administrativo da plataforma. Possuem maior nível de privilégio, sendo encarregados do controle de acesso, cadastro de usuários, definição de perfis e acompanhamento geral do funcionamento do sistema. Essa classe utiliza o sistema com frequência elevada e exige acesso ampliado às funções administrativas e de supervisão.

b) Profissionais de saúde

Incluem médicos, enfermeiros e demais profissionais envolvidos no atendimento hospitalar. São os principais usuários operacionais do sistema, utilizando-o para consultar históricos, registrar informações no prontuário eletrônico, realizar teletriagens, avaliar sintomas, classificar riscos e se comunicar com outros profissionais. Essa é a classe mais importante para o funcionamento do produto, pois concentra a maior parte das funções essenciais previstas.

c) Atendentes ou equipe de recepção

São usuários responsáveis principalmente pelo cadastro e atualização de dados dos pacientes, consulta de informações básicas e apoio ao fluxo inicial de atendimento. Seu uso do sistema está mais concentrado nas funcionalidades de gestão de pacientes, com nível de acesso menor que o dos profissionais de saúde, especialmente em relação ao conteúdo clínico detalhado.

d) Gestores hospitalares

Correspondem aos usuários que utilizam a plataforma para acompanhamento geral das informações do sistema, visualização de dashboards, relatórios e apoio à organização dos processos hospitalares. Seu acesso tende a ser mais voltado à consulta e supervisão, com menor participação na inserção de dados clínicos diretamente.

e) Pacientes

Os pacientes são entidades centrais do sistema e podem ser considerados usuários indiretos da plataforma, pois seus dados são cadastrados, consultados e acompanhados por outros perfis de usuário. Embora o documento não detalhe explicitamente um acesso direto do paciente ao sistema principal, essa classe é fundamental dentro do domínio do produto, uma vez que todo o fluxo de prontuário, triagem e monitoramento está relacionado a ela.

De forma geral, as classes mais importantes para a satisfação dos objetivos do produto são os **profissionais de saúde** e os **administradores do sistema**, pois são eles que fazem uso mais intenso das funções essenciais da plataforma. Em seguida, destacam-se os **atendentes/recepção** e os **gestores hospitalares**, que possuem participação complementar no uso do sistema. Já os **pacientes**, embora centrais no domínio da aplicação, não aparecem no documento como usuários diretos da interface principal.

2.4 Ambiente de Trabalho

O sistema **HealthSys Distribuído** será executado em um ambiente distribuído baseado em containers, utilizando **Docker** para a containerização dos serviços e **Kubernetes** para orquestração, escalabilidade e gerenciamento da aplicação. O backend será desenvolvido com **Java 17+**, utilizando **Spring Boot**, enquanto o frontend será desenvolvido em **React**, com necessidade de **Node.js** para sua execução e gerenciamento de dependências.

No que se refere ao armazenamento e processamento de dados, o sistema utilizará o **PostgreSQL** como banco de dados principal para persistência das informações da aplicação. Para a comunicação assíncrona entre os serviços distribuídos, será adotado o **Apache Kafka** como mecanismo de mensageria. Para fins de monitoramento e observabilidade, o ambiente contará com **Prometheus** e **Grafana**, bem como com a **ELK Stack** para centralização e análise de logs.

Quanto à execução do sistema, os principais pré-requisitos incluem a instalação de **Docker**, **Kubernetes**, **Java 17+** e **Node.js**, permitindo que a aplicação seja executada em ambiente de desenvolvimento por meio de interface web acessada localmente.

Em relação à plataforma de hardware e ao sistema operacional, não foram definidos requisitos específicos. Dessa forma, entende-se que o sistema poderá ser executado em ambientes compatíveis com as tecnologias adotadas, desde que ofereçam suporte à execução de containers, ao banco de dados PostgreSQL, ao serviço de mensageria Kafka e aos componentes de monitoramento previstos na arquitetura.

2.5 Design e Implementação Restrições

O desenvolvimento do **HealthSys Distribuído** deverá seguir uma arquitetura distribuída baseada em **microservices**, de forma a atender à proposta central do projeto e aos conteúdos previstos na disciplina de **Computação Distribuída**. Dessa forma, as decisões de design e implementação devem preservar a separação de responsabilidades entre os serviços, a comunicação entre componentes e a possibilidade de escalabilidade horizontal e alta disponibilidade.

Como restrição tecnológica, o projeto adotará **React** no frontend e **Spring Boot** no backend, mantendo a implementação alinhada à arquitetura originalmente proposta. Além disso, o sistema deverá utilizar **PostgreSQL** como banco de dados principal, **Kafka** como mecanismo de comunicação assíncrona entre serviços e **JWT** para autenticação e controle de acesso. Essas definições limitam a adoção de outras tecnologias fora da stack escolhida pela equipe durante o desenvolvimento.

No que se refere à infraestrutura, a aplicação deverá ser executada em ambiente baseado em **Docker** para containerização e **Kubernetes** para orquestração e gerenciamento dos serviços. Isso impõe aos desenvolvedores a necessidade de organizar a aplicação de forma compatível com execução em containers, deploy distribuído e integração entre múltiplos serviços.

Em relação aos protocolos e mecanismos de comunicação, o projeto deverá contemplar **comunicação síncrona via API REST** e **comunicação assíncrona por mensageria**, utilizando **Kafka**. Além disso, o uso de **API Gateway**, **Service Discovery** e **Message Brokers** faz parte da proposta arquitetural, o que restringe o desenvolvimento a um modelo distribuído com integração formal entre serviços, em vez de uma solução monolítica.

Também constituem restrições de implementação os requisitos relacionados à observabilidade e à confiabilidade do sistema. O projeto prevê a utilização de **Prometheus** e **Grafana** para monitoramento, **ELK Stack** para centralização de logs, além de mecanismos voltados à tolerância a falhas, como **health checks com Spring Actuator** e simulação de falhas por meio de **Chaos Engineering**. Assim, a implementação deverá considerar recursos mínimos de monitoramento, registro de eventos e acompanhamento da saúde dos serviços distribuídos.

Outra restrição relevante é o prazo de desenvolvimento estabelecido no cronograma do projeto, que prevê execução em **8 semanas**, distribuídas entre planejamento, implementação dos serviços básicos, integração distribuída e deploy com monitoramento. Essa limitação impõe à equipe a necessidade de priorizar as funcionalidades essenciais do sistema e manter o desenvolvimento concentrado nos componentes não classificados como opcionais no documento.

Por fim, como o documento não define restrições específicas de hardware ou sistema operacional, entende-se que a principal limitação de ambiente está na necessidade de compatibilidade com os pré-requisitos de execução do projeto, especialmente **Docker**, **Kubernetes**, **Java 17+** e **Node.js**, bem como com os serviços complementares exigidos pela arquitetura distribuída adotada.

2.6 Documentação do Usuário

A documentação do usuário do **HealthSys Distribuído** será composta por materiais de apoio destinados a orientar a utilização das principais funcionalidades da plataforma, especialmente as operações relacionadas ao cadastro de pacientes, consulta de histórico, uso do prontuário eletrônico, teletriagem, visualização de relatórios e acompanhamento das notificações do sistema. Essa documentação será entregue em conjunto com o software, em conformidade com a previsão de **documentação final** e **vídeo de demonstração** presente no cronograma do projeto.

Os componentes previstos da documentação do usuário são:

- **Manual do usuário**, contendo instruções de acesso ao sistema, descrição das telas principais e orientações para utilização das funcionalidades essenciais;
- **Guia rápido de uso**, com um resumo das operações básicas da plataforma para consulta rápida;
- **Tutorial de acesso e navegação**, apresentando o fluxo inicial de uso do sistema, desde autenticação até execução das funções principais;
- **Vídeo de demonstração**, com apresentação prática da plataforma e de seus principais recursos;
- **Guia de execução do sistema em ambiente local**, contendo orientações básicas para inicialização da aplicação em ambiente de desenvolvimento.

Quanto aos formatos de entrega, a documentação poderá ser disponibilizada nos seguintes meios:

- **PDF**, para o manual do usuário e o guia rápido;
- **Documento digital em texto**, podendo ser entregue em formato como Markdown ou documento editável;
- **Vídeo**, para demonstração prática do sistema;
- **Repositório do projeto**, no qual poderão constar instruções complementares de execução e uso da aplicação.

De forma geral, a documentação do usuário terá como objetivo facilitar a compreensão e a utilização do **HealthSys Distribuído**, servindo como apoio tanto para apresentação acadêmica do projeto quanto para a orientação dos usuários sobre as funcionalidades implementadas na plataforma.

Tem um ponto importante: o PDF **não detalha nominalmente** "manual do usuário", "guia rápido" e "tutorial"; ele só explicita que haverá **documentação final** e **vídeo de demonstração**. Então essa lista acima funciona como uma **formalização adequada para o SRS**, derivada da entrega prevista no cronograma.

2.7 Suposições e Dependências

O desenvolvimento do HealthSys Distribuído parte da suposição de que a equipe terá acesso a um ambiente compatível com a execução de aplicações distribuídas baseadas em containers, incluindo suporte a Docker, Kubernetes, Java 17+ e Node.js, uma vez que esses componentes são apresentados como pré-requisitos para execução do projeto. Também se assume que o sistema será desenvolvido e testado em ambiente capaz de suportar a arquitetura de microservices definida no planejamento.

Assume-se ainda que a implementação seguirá a arquitetura proposta no projeto, com separação entre frontend e backend distribuído, utilização de React no frontend, Spring Boot no backend, PostgreSQL como banco de dados principal e Kafka como mecanismo de comunicação assíncrona entre os serviços. Dessa forma, os requisitos estabelecidos neste documento dependem da manutenção dessas decisões tecnológicas ao longo do desenvolvimento, pois alterações significativas na stack poderão impactar a implementação, a integração entre os componentes e o cronograma previsto.

Outra suposição relevante é a de que os serviços da aplicação poderão se comunicar corretamente por meio dos mecanismos definidos na arquitetura, especialmente API Gateway, API REST e mensageria assíncrona. Também se pressupõe que os componentes de infraestrutura e observabilidade, como Redis, Prometheus, Grafana e ELK Stack, estarão disponíveis para integração ao sistema, conforme previsto no documento do projeto.

No que se refere às dependências, o projeto depende diretamente de tecnologias e ferramentas externas para sua implementação e operação, incluindo Docker e Kubernetes para containerização e orquestração, PostgreSQL para persistência de dados, Kafka para mensageria, JWT para autenticação e controle de acesso, além de Prometheus, Grafana e ELK Stack para monitoramento e centralização de logs. Essas dependências são essenciais para que a solução mantenha as características de sistema distribuído previstas no planejamento.

O projeto também depende da correta implementação dos serviços principais previstos no cronograma, como serviço de usuários, serviço de pacientes, autenticação, API REST, API Gateway, serviço de triagem, notificações assíncronas e cache distribuído. Como o desenvolvimento foi planejado para ocorrer em 8 semanas, qualquer atraso na disponibilização desses componentes ou dificuldade na integração entre eles poderá afetar a entrega final do sistema.

Além disso, assume-se que os recursos opcionais do projeto não farão parte da entrega principal, uma vez que o desenvolvimento está sendo conduzido com foco apenas nos itens essenciais do documento. Assim, funcionalidades opcionais, bem como dependências associadas a esses módulos, não são consideradas obrigatórias para o cumprimento dos requisitos definidos neste SRS.

3. Requisitos do Sistema

3.1 Requisitos Funcionais

ID	INTERESSADO	DESCRIÇÃO
RF01	Atendentes / Recepção	O sistema deve permitir o cadastro de pacientes com informações pessoais e de contato.
RF02	Atendentes / Recepção	O sistema deve permitir a consulta e atualização do histórico cadastral do paciente.
RF03	Profissionais de Saúde	O sistema deve permitir o registro de informações no prontuário eletrônico do paciente.
RF04	Profissionais de Saúde	O sistema deve permitir o registro de consultas, exames e medicamentos no prontuário.
RF05	Profissionais de Saúde	O sistema deve permitir o registro de alergias e vacinas associadas ao paciente.
RF06	Profissionais de Saúde	O sistema deve permitir a realização de teletriagem com avaliação de sintomas do paciente.
RF07	Profissionais de Saúde	O sistema deve permitir a classificação de risco durante a triagem.
RF08	Profissionais de Saúde	O sistema deve permitir a visualização do prontuário eletrônico e do histórico clínico do paciente.
RF09	Profissionais de Saúde	O sistema deve permitir a comunicação entre profissionais de saúde dentro da plataforma.
RF10	Gestores Hospitalares	O sistema deve disponibilizar dashboard hospitalar para acompanhamento das informações do sistema.
RF11	Administradores do Sistema	O sistema deve permitir autenticação de usuários e controle de acesso por perfil.
RF12	Administradores / Profissionais de Saúde	O sistema deve enviar notificações e alertas relacionados às atividades da plataforma.

3.2 Requisitos Não Funcionais

ID	INTERESSADO	DESCRIÇÃO
RNF01	Equipe de Desenvolvimento	O sistema deve ser desenvolvido com arquitetura distribuída baseada em microservices.
RNF02	Equipe de Desenvolvimento	O frontend do sistema deve ser implementado em React.
RNF03	Equipe de Desenvolvimento	O backend do sistema deve ser implementado em Java 17+ com Spring Boot.
RNF04	Equipe de Desenvolvimento / DBA	O sistema deve utilizar PostgreSQL como banco de dados principal.
RNF05	Equipe de Desenvolvimento	O sistema deve utilizar Kafka para comunicação assíncrona entre serviços distribuídos.
RNF06	Equipe de Infraestrutura	Os serviços da aplicação devem ser executados em containers Docker.
RNF07	Equipe de Infraestrutura	O ambiente da aplicação deve ser compatível com orquestração via Kubernetes.
RNF08	Administradores do Sistema	O sistema deve possuir autenticação segura baseada em JWT.
RNF09	Equipe de Infraestrutura	O sistema deve possibilitar monitoramento por meio de Prometheus e Grafana.
RNF10	Equipe de Infraestrutura	O sistema deve permitir centralização e análise de logs por meio da ELK Stack.
RNF11	Usuários do Sistema	A aplicação deve estar disponível por interface web em ambiente de desenvolvimento e execução.
RNF12	Equipe de Desenvolvimento	O sistema deve suportar comunicação síncrona via API REST entre os componentes necessários.

3.3 Regras de Negócio

ID	INTERESSADO	DESCRIÇÃO
RN01	Atendentes / Recepção	Todo atendimento deve estar associado a um paciente previamente cadastrado no sistema.
RN02	Profissionais de Saúde	Todo prontuário eletrônico deve estar vinculado a um paciente específico.
RN03	Profissionais de Saúde	Toda triagem deve registrar obrigatoriamente a avaliação de sintomas e a classificação de risco do paciente.
RN04	Administradores do Sistema	O acesso às funcionalidades do sistema deve respeitar o perfil de usuário autenticado.
RN05	Profissionais de Saúde	Apenas usuários autorizados devem poder registrar ou alterar informações clínicas do prontuário.
RN06	Gestores Hospitalares	Relatórios e dashboards devem utilizar dados já registrados pelos módulos operacionais do sistema.
RN07	Profissionais de Saúde	O histórico clínico do paciente deve permanecer disponível para consulta ao longo dos atendimentos registrados.
RN08	Administradores / Profissionais de Saúde	Notificações do sistema devem estar relacionadas a eventos e ações realizadas na plataforma.

4. Casos de Uso

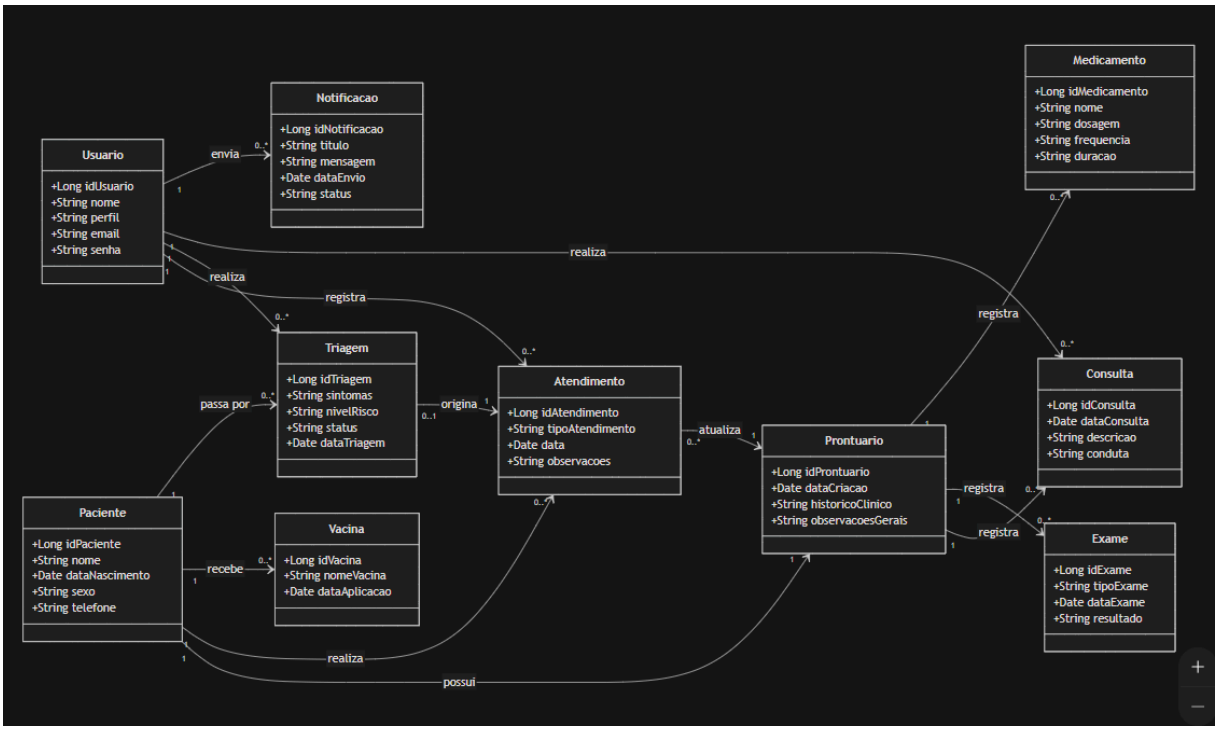
4.1 Atores

ID	ATOR	DESCRIÇÃO
AT01	Administrador do Sistema	Responsável pelo gerenciamento de usuários, controle de acesso, autenticação e supervisão geral da plataforma.
AT02	Atendente / Recepção	Responsável pelo cadastro de pacientes, consulta de histórico cadastral e apoio ao fluxo inicial de atendimento.
AT03	Profissional de Saúde	Responsável por registrar prontuários, consultas, exames, medicamentos, realizar triagens e acessar informações clínicas dos pacientes.
AT04	Gestor Hospitalar	Responsável por acompanhar dashboards, relatórios e informações consolidadas do sistema para apoio à gestão.

4.2 Casos de Uso

ID	ATOR	CASO DE USO
UC01	Administrador do Sistema	Autenticar usuário no sistema
UC02	Administrador do Sistema	Gerenciar usuários e perfis de acesso
UC03	Atendente / Recepção	Cadastrar paciente
UC04	Atendente / Recepção	Consultar histórico do paciente
UC05	Profissional de Saúde	Registrar prontuário eletrônico
UC06	Profissional de Saúde	Registrar consultas, exames e medicamentos
UC07	Profissional de Saúde	Registrar alergias e vacinas
UC08	Profissional de Saúde	Realizar teletriagem
UC09	Profissional de Saúde	Avaliar sintomas e classificar risco
UC10	Profissional de Saúde	Visualizar prontuário e histórico clínico
UC11	Profissional de Saúde	Comunicar-se com outros profissionais
UC12	Gestor Hospitalar	Visualizar dashboard hospitalar e relatórios
UC13	Administrador do Sistema / Profissional de Saúde	Receber notificações e alertas do sistema

5. Diagrama de Classe



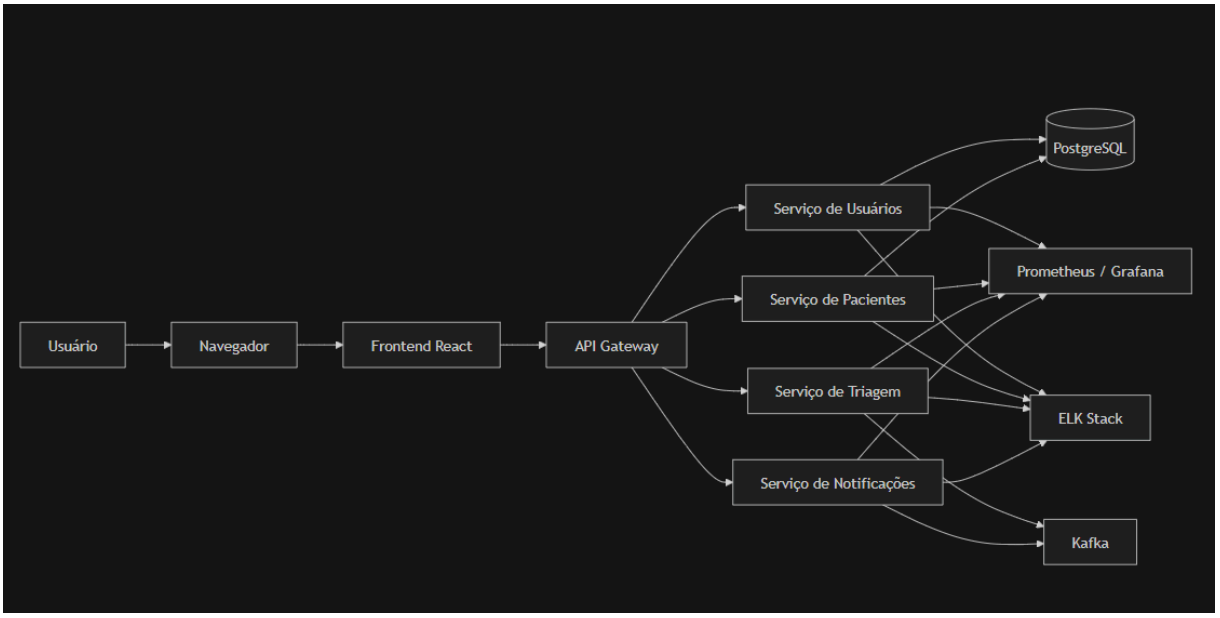
6. Arquitetura Sistemas Operacionais

O HealthSys Distribuído foi projetado para operar em um ambiente distribuído compatível com sistemas operacionais modernos, priorizando a execução dos serviços em plataformas com suporte à conteinerização e orquestração. Para a infraestrutura da aplicação, considera-se preferencialmente a utilização de ambientes baseados em Linux, devido à ampla compatibilidade com Docker, Kubernetes, PostgreSQL, Kafka e demais componentes da arquitetura proposta.

No contexto de desenvolvimento, o sistema poderá ser implementado e testado em diferentes sistemas operacionais, como Windows, Linux e macOS, desde que o ambiente disponibilize suporte aos pré-requisitos definidos para o projeto, especialmente Java 17+, Node.js, Docker e Kubernetes. Dessa forma, a arquitetura operacional do sistema mantém portabilidade entre diferentes plataformas utilizadas pela equipe de desenvolvimento.

Como o HealthSys Distribuído será disponibilizado como uma aplicação web, o acesso pelos usuários finais ocorrerá por meio de navegadores compatíveis, independentemente do sistema operacional utilizado na máquina cliente. Assim, o frontend da aplicação será executado no navegador do usuário, enquanto os serviços de backend, banco de dados, mensageria e monitoramento operarão em containers no ambiente de execução da plataforma.

Essa organização permite padronização do ambiente, maior facilidade de implantação e melhor compatibilidade entre os diferentes componentes do sistema, garantindo que a solução possa ser executada de forma consistente em diferentes contextos operacionais.



7. Apêndice A: Glossário

API (Application Programming Interface): conjunto de regras e definições que permite a comunicação entre diferentes sistemas ou componentes de software.

API Gateway: componente responsável por centralizar o acesso aos serviços do sistema, realizando o roteamento das requisições para os microservices adequados.

Backend: camada do sistema responsável pelo processamento das regras de negócio, comunicação com banco de dados e execução dos serviços da aplicação.

Cache: mecanismo de armazenamento temporário de dados com o objetivo de melhorar o desempenho do sistema.

Container: unidade padronizada de software que empacota aplicação, dependências e configurações, permitindo sua execução de forma consistente em diferentes ambientes.

Dashboard: painel visual que apresenta informações resumidas, indicadores e dados relevantes do sistema para acompanhamento e supervisão.

Docker: plataforma utilizada para containerização dos serviços da aplicação.

ELK Stack: conjunto de ferramentas formado por Elasticsearch, Logstash e Kibana, utilizado para centralização, análise e visualização de logs do sistema.

Frontend: camada do sistema responsável pela interface com o usuário, permitindo interação com as funcionalidades da aplicação.

HealthSys Distribuído: nome do sistema especificado neste documento, proposto como uma plataforma de gestão hospitalar distribuída.

HTTP: protocolo de comunicação utilizado para troca de informações entre cliente e servidor em aplicações web.

JWT (JSON Web Token): padrão utilizado para autenticação e controle de acesso, permitindo a identificação segura de usuários no sistema.

Kafka: plataforma de mensageria e streaming de eventos utilizada para comunicação assíncrona entre os serviços do sistema.

Kubernetes: ferramenta de orquestração de containers utilizada para gerenciamento, implantação e escalabilidade dos serviços da aplicação.

Microservices: arquitetura de software em que a aplicação é dividida em serviços independentes, cada um responsável por uma funcionalidade específica.

Monitoramento: processo de acompanhamento do funcionamento do sistema, incluindo métricas, desempenho, disponibilidade e falhas.

Node.js: ambiente de execução utilizado no desenvolvimento e gerenciamento de dependências do frontend React.

Observabilidade: capacidade de analisar o comportamento interno do sistema a partir de métricas, logs e rastreamentos.

PostgreSQL: sistema gerenciador de banco de dados relacional adotado como banco de dados principal do projeto.

Prontuário Eletrônico: registro digital das informações clínicas do paciente, incluindo histórico, consultas, exames e medicamentos.

React: biblioteca JavaScript utilizada para desenvolvimento da interface web do sistema.

Redis: tecnologia utilizada como cache distribuído para apoio ao desempenho da aplicação.

REST API: estilo de arquitetura para construção de serviços web baseado em requisições HTTP, utilizado para comunicação síncrona entre componentes do sistema.

SRS (Software Requirements Specification): documento de especificação de requisitos de software, no qual são descritas as funcionalidades, restrições e características esperadas do sistema.

Spring Boot: framework Java utilizado para implementação dos serviços de backend do sistema.

Teletriagem: processo de avaliação inicial do paciente, com análise de sintomas e classificação de risco, realizado com apoio do sistema.

Triagem: etapa de avaliação inicial do paciente para identificar sintomas, prioridade e nível de risco no atendimento.

8. Apêndice B: Modelos de Análise

Este apêndice apresenta os principais modelos de análise elaborados para apoiar a especificação do **HealthSys Distribuído**, contribuindo para a compreensão estrutural, funcional e arquitetural do sistema.

Os modelos de análise considerados neste documento são:

a) Diagrama de Casos de Uso
Representa os principais atores do sistema e suas interações com as funcionalidades essenciais da plataforma, como autenticação, cadastro de pacientes, consulta de histórico, registro de prontuário eletrônico, teletriagem, classificação de risco, visualização de relatórios e recebimento de notificações.

b) Diagrama de Classes
Apresenta a estrutura conceitual das principais entidades do sistema, incluindo classes como **Usuário**, **Paciente**, **Prontuário**, **Atendimento**, **Triagem**, **Vacina**, **Consulta**, **Exame**, **Medicamento** e **Notificação**, bem como seus relacionamentos.

c) Diagrama da Arquitetura Operacional
Descreve a organização da solução em ambiente distribuído, contemplando **frontend em React**, **API Gateway**, **serviços em Spring Boot**, **PostgreSQL**, **Kafka**, além dos componentes de infraestrutura e observabilidade, como **Docker**, **Kubernetes**, **Prometheus**, **Grafana** e **ELK Stack**.

d) Modelo Entidade-Relacionamento (conceitual)
Representa a organização dos principais dados do sistema, com foco nas entidades fundamentais do domínio hospitalar, como pacientes, usuários, atendimentos, triagens e registros clínicos.

e) Tabelas de Requisitos Funcionais, Não Funcionais e Regras de Negócio

Consolidam os requisitos levantados para o sistema, organizando as funcionalidades essenciais, características de qualidade e restrições que orientam o desenvolvimento da aplicação.

Esses modelos complementam a especificação textual presente neste SRS e auxiliam na visualização dos componentes, fluxos e estruturas relevantes do **HealthSys Distribuído**, servindo como apoio ao entendimento, à implementação e à validação do projeto.

9. Aprovação

_____ ___/___/___

Cliente

Nome:

_____ ___/___/___

Gerente do Projeto

Nome: